Senior Backend Engineer — Stage 1 Technical Assessment

CubeSquare Platform · Node.js / TypeScript

Contents

Pre-Screening **1**

Question 1 — Third-Party Integration Architecture **2**
1.1 OAuth 2.0 Token Manager Service . . . 2
1.2 KYC Webhook Handler . . . 4
1.3 Retry Wrapper for Outbound KYC API Calls . . . 6

Question 2 — Multi-Tenant Authorization at the Application Layer **10**
2.1 Tenant Isolation Architecture . . . 10
2.2 TypeScript Tenant Isolation Middleware . . . 10
2.3 Cross-Tenant Access Error Handling . . . 12
2.4 Attack Vector — Insecure Direct Object Reference (IDOR) . . . 13

Question 3 — Engineering Judgment: Fireblocks Integration **15**
3.1 Implementation Plan for the Remaining Three Days . . . 15
3.2 Data Required for Transaction Reconciliation . . . 16
3.3 Stakeholder Communication . . . 17

AI Usage Review **20**

Pre-Screening

#	Question	Answer
1	Do you have 4+ years of professional backend engineering experience with Node.js and TypeScript in production?	**YES**
2	Have you built production integrations with external APIs involving OAuth/token management, webhook handling, and retry/failure-recovery logic?	**YES**
3	Have you implemented idempotency for a transactional API or asynchronous workflow in production?	**YES**

GitHub profile: [Add GitHub URL]

Most complex third-party integration: [Add one sentence describing the API, what made the integration complex, and how failures were handled.]

Question 1 — Third-Party Integration Architecture

CubeSquare integrates with a KYC provider. Requirements:

- OAuth 2.0 client credentials flow (tokens expire after 60 minutes)
- KYC status changes delivered as webhooks
- Rate limit: 100 requests/minute (HTTP 429 on exceed)
- Webhooks are not guaranteed-once (duplicates possible)
- Payload: `event_id` (UUID), `investor_id`, `new_status`, `timestamp`

1.1 OAuth 2.0 Token Manager Service

Assumptions

- Provider supports OAuth 2.0 client credentials flow
- Token endpoint returns `access_token` and `expires_in`
- Axios for HTTP requests
- Credentials from environment variables
- Service instantiated once per application process

Implementation — `oauth-token.service.ts`

```ts
import axios from "axios";

export class OAuthTokenService {
  private accessToken: string | null = null;
  private expiresAt = 0;
  private refreshPromise: Promise<string> | null = null;

  // Refresh five minutes before the provider's token expiry.
  private readonly refreshBuffer = 5 * 60 * 1000;

  async getAccessToken(): Promise<string> {
    // Return the cached token while it is safely within its validity period.
    if (
      this.accessToken &&
      Date.now() < this.expiresAt - this.refreshBuffer
    ) {
      return this.accessToken;
    }

    // Reuse an in-flight refresh request to avoid concurrent token requests.
    if (this.refreshPromise) {
      return this.refreshPromise;
    }

    this.refreshPromise = this.fetchToken();

    try {
      return await this.refreshPromise;
    } finally {
      this.refreshPromise = null;
    }
  }
```

```
  private async fetchToken(): Promise<string> {
    try {
      const response = await axios.post(
        process.env.KYC_TOKEN_URL!,
        new URLSearchParams({
          grant_type: "client_credentials",
          client_id: process.env.KYC_CLIENT_ID!,
          client_secret: process.env.KYC_CLIENT_SECRET!,
        }),
        {
          headers: {
            "Content-Type": "application/x-www-form-urlencoded",
          },
          timeout: 10000,
        }
      );

      const { access_token, expires_in } = response.data;

      if (
        typeof access_token !== "string" ||
        !access_token ||
        typeof expires_in !== "number" ||
        !Number.isFinite(expires_in) ||
        expires_in <= 0
      ) {
        throw new Error("Invalid OAuth token response");
      }

      this.accessToken = access_token;
      this.expiresAt = Date.now() + expires_in * 1000;

      return access_token;
    } catch (error) {
      // Never log credentials or access tokens.
      console.error("KYC token refresh failed:", {
        message: axios.isAxiosError(error)
          ? error.message
          : "Unknown authentication error",
      });

      throw new Error(
        "Unable to obtain access token from KYC provider"
      );
    }
  }

  // Can be called when the provider rejects a token with HTTP 401.
  invalidateToken(): void {
    this.accessToken = null;
    this.expiresAt = 0;
  }
}
```

Engineering Decisions

- **In-memory caching** — Avoids unnecessary OAuth requests and reduces latency.
- **Refresh buffer** — Refreshes five minutes before expiry to reduce risk of using an expired token.
- **Concurrency control** — `refreshPromise` ensures simultaneous requests share one token-fetch operation.
- **Failure handling** — A failed refresh throws rather than returning an expired token.
- **Invalidation** — `invalidateToken()` clears the cached token after an authentication failure.

For multiple Node.js instances, each process has its own cache. A distributed cache or centralized token service can be considered if provider token issuance policies or rate limits require coordination.

1.2 KYC Webhook Handler

Assumptions

- Express for the API
- Zod for payload validation
- `event_id` has a unique constraint in the event repository
- Durable background queue (e.g. BullMQ) available
- Repository, transaction, and queue operations implemented separately

Implementation — `kyc-webhook.service.ts`

```ts
import { Request, Response } from "express";
import { z } from "zod";

const KycWebhookSchema = z.object({
  event_id: z.string().uuid(),
  investor_id: z.string().min(1),
  new_status: z.enum([
    "approved",
    "rejected",
    "pending",
  ]),
  timestamp: z.string().datetime({ offset: true }),
});

export class KycWebhookService {
  async handleWebhook(
    req: Request,
    res: Response
  ): Promise<void> {
    // 1. Validate the incoming webhook payload.
    const validation = KycWebhookSchema.safeParse(req.body);

    if (!validation.success) {
      res.status(400).json({
        success: false,
        message: "Invalid webhook payload",
        errors: validation.error.flatten(),
      });
      return;
    }
```

```
  const event = validation.data;

  try {
    // 2. Atomically insert the event using its unique event_id.
    // Returns false when the event has already been received.
    const isNewEvent =
      await webhookRepository.createIfNotExists(event);

    if (!isNewEvent) {
      res.status(200).json({
        success: true,
        message: "Duplicate event ignored",
      });
      return;
    }

    // 3. Enqueue the event for asynchronous processing.
    await webhookQueue.add(
      "process-kyc-event",
      { eventId: event.event_id },
      {
        jobId: event.event_id,
        attempts: 5,
        backoff: {
          type: "exponential",
          delay: 2000,
        },
      }
    );

    // 4. Acknowledge only after durable queue acceptance.
    res.status(200).json({
      success: true,
      message: "Webhook accepted successfully",
    });
  } catch (error) {
    console.error("KYC webhook ingestion failed:", error);

    res.status(500).json({
      success: false,
      message: "Failed to accept webhook",
    });
  }
}

// Called by the background worker.
async processKycEvent(eventId: string): Promise<void> {
  const event =
    await webhookRepository.findByEventId(eventId);

  if (!event) {
    throw new Error(`Webhook event not found: ${eventId}`);
  }
```

```
    // Atomically claim the event to prevent concurrent processing.
    const claimed =
      await webhookRepository.claimForProcessing(eventId);

    if (!claimed) {
      return;
    }

    try {
      // Update investor status and mark the event processed
      // within the same database transaction.
      await database.transaction(async (tx) => {
        await investorRepository.updateKycStatus(
          event.investor_id,
          event.new_status,
          tx
        );

        await webhookRepository.markAsProcessed(eventId, tx);
      });
    } catch (error) {
      await webhookRepository.markAsFailed(eventId);
      throw error; // Allows the queue retry mechanism to execute.
    }
  }
}
```

Engineering Decisions

- Zod validates the incoming payload at runtime.
- Unique `event_id` prevents duplicate event registration.
- Durable queue keeps business processing outside the HTTP request lifecycle.
- Duplicate deliveries receive HTTP 200 (event already accepted).
- Queue/persistence failures return HTTP 500 so the provider can retry.
- Background processing uses an atomic event claim.
- Investor status update + event completion occur in one database transaction.
- Failed jobs are retried through the queue.

Reliability note: Repository and queue must provide durable persistence and atomic operations. A transactional outbox is a stronger production option to avoid losing an event between DB persistence and queue enqueueing.

1.3 Retry Wrapper for Outbound KYC API Calls

Assumptions

- Axios for outbound HTTP
- Provider returns HTTP 429 on rate limit
- Provider may include `Retry-After` header
- Only HTTP 429 responses are retried
- Maximum 5 retries (excluding initial request)

Implementation — `kyc-retry.service.ts`

```ts
import axios from "axios";

export class KycRetryExhaustedError extends Error {
  constructor(
    public readonly maxRetries: number,
    public readonly statusCode: number = 429
  ) {
    super(
      `KYC API request failed after ${maxRetries} retry attempts`
    );
    this.name = "KycRetryExhaustedError";
  }
}

export class KycRetryService {
  private readonly maxRetries = 5;
  private readonly baseDelay = 1000;
  private readonly maxDelay = 30000;

  async execute<T>(request: () => Promise<T>): Promise<T> {
    let attempt = 0;

    while (true) {
      try {
        return await request();
      } catch (error) {
        const isRateLimited =
          axios.isAxiosError(error) &&
          error.response?.status === 429;

        // Propagate errors that are not rate-limit responses.
        if (!isRateLimited) {
          throw error;
        }

        // Stop after the configured number of retries.
        if (attempt >= this.maxRetries) {
          throw new KycRetryExhaustedError(
            this.maxRetries,
            429
          );
        }

        attempt++;

        const retryAfter = this.getRetryAfter(error);

        // Exponential backoff, capped at maxDelay.
        const exponentialDelay = Math.min(
          this.baseDelay * Math.pow(2, attempt - 1),
          this.maxDelay
        );
```

```
        const delay = Math.min(
          retryAfter ?? exponentialDelay,
          this.maxDelay
        );

        console.warn(
          `KYC API rate limited. Retry ${attempt}/${this.maxRetries} in ${delay}ms`
        );

        await this.sleep(delay);
      }
    }
  }

  private getRetryAfter(error: unknown): number | null {
    if (!axios.isAxiosError(error)) {
      return null;
    }

    const header = error.response?.headers?.["retry-after"];

    if (!header) {
      return null;
    }

    const seconds = Number(header);

    if (Number.isFinite(seconds) && seconds >= 0) {
      return seconds * 1000;
    }

    const retryDate = Date.parse(header);

    if (!Number.isNaN(retryDate)) {
      return Math.max(0, retryDate - Date.now());
    }

    return null;
  }

  private sleep(ms: number): Promise<void> {
    return new Promise((resolve) => setTimeout(resolve, ms));
  }
}
```

Usage Example

```
const retryService = new KycRetryService();

async function getInvestorDetails(investorId: string) {
  return retryService.execute(async () => {
    const token = await tokenManager.getAccessToken();
```

```
    return axios.get(
      `https://api.kyc-provider.com/investors/${investorId}`,
      {
        headers: {
          Authorization: `Bearer ${token}`,
        },
        timeout: 10000,
      }
    );
  });
}
```

Engineering Decisions

- **Exponential backoff** — Reduces pressure on the provider during rate-limit periods.
- **Retry-After support** — Respects the provider's suggested retry window.
- **Maximum retries** — Prevents infinite retry loops.
- **Typed error** — `KycRetryExhaustedError` distinguishes exhausted rate-limit retries from other failures.
- **Selective retries** — Only HTTP 429 is retried; other errors propagate immediately.
- **Token management** — Token is retrieved inside the request callback so a refreshed token can be supplied when needed.

For multiple application instances, a Redis-backed distributed rate limiter should additionally coordinate requests against the provider's global limit of 100 requests per minute.

Question 2 — Multi-Tenant Authorization at the Application Layer

CubeSquare serves multiple licensed operator-tenants. An investor authenticated under Tenant A must never be able to read, modify, or transact against data belonging to Tenant B — even with a valid JWT.

This question is about the **application layer only**. Database schema design is owned by the Senior PostgreSQL Database Engineer.

2.1 Tenant Isolation Architecture

I would implement a JWT-based tenant context middleware combined with service-layer authorization to enforce tenant isolation across all authenticated API requests.

The middleware establishes the authenticated user's tenant context, while the service layer ensures that every resource access is scoped to that tenant.

Request Lifecycle

1. **Authentication** — Validate JWT signature, expiration, issuer, and audience.
2. **Tenant context extraction** — Extract `tenant_id` and `investor_id` from verified JWT claims.
3. **Tenant context attachment** — Attach validated tenant and investor identities to the request context.
4. **Service-layer authorization** — Before accessing a resource, verify it belongs to the authenticated tenant and that the investor is authorized.
5. **Business operation** — Execute read/update/transaction only after authorization succeeds.

Enforcement Strategy

- Never trust `tenant_id` or `investor_id` supplied via request body, query, or URL parameters as the source of identity.
- Derive tenant identity exclusively from the verified JWT.
- Pass tenant context explicitly to service methods.
- Apply resource-level ownership checks for investors, properties, and transactions.
- Ensure every authenticated endpoint uses the authentication middleware.
- Perform authorization checks before any business operation, especially financial transactions.

The middleware provides the initial security boundary; service-level checks prevent authorization bypass through internal service calls or incorrectly scoped endpoints.

2.2 TypeScript Tenant Isolation Middleware

Assumptions

- Express API
- JWT authentication middleware already verifies signature and expiration
- Verified claims available via `req.user`
- Repository methods support tenant-scoped resource lookup
- Centralized logger available

Implementation — `tenant-isolation.middleware.ts`

```
import { Request, Response, NextFunction } from "express";

interface JwtClaims {
  tenant_id: string;
```

```
  investor_id: string;
}

interface TenantContext {
  tenantId: string;
  investorId: string;
}

interface AuthenticatedRequest extends Request {
  user?: JwtClaims;
  tenantContext?: TenantContext;
}

export const tenantIsolationMiddleware = (
  req: AuthenticatedRequest,
  res: Response,
  next: NextFunction
): void => {
  const user = req.user;

  // Ensure the authenticated user has valid tenant claims.
  if (!user?.tenant_id || !user?.investor_id) {
    res.status(401).json({
      success: false,
      message: "Invalid authentication context",
      code: "INVALID_AUTH_CONTEXT",
    });
    return;
  }

  // Establish trusted tenant context from verified JWT claims.
  req.tenantContext = {
    tenantId: user.tenant_id,
    investorId: user.investor_id,
  };

  next();
};

export const validateTenantOwnership = (
  resourceTenantId: string,
  tenantContext: TenantContext
): boolean => {
  return resourceTenantId === tenantContext.tenantId;
};
```

Example Service-Level Usage

```
async function getInvestor(
  req: AuthenticatedRequest,
  investorId: string
) {
  const context = req.tenantContext!;
```

```
  // Fetch the resource within the authenticated tenant.
  const investor = await investorRepository.findByIdAndTenantId(
    investorId,
    context.tenantId
  );

  if (!investor) {
    throw new ResourceNotFoundError("Investor not found");
  }

  // Investors can only access their own investor record.
  if (investor.id !== context.investorId) {
    throw new ResourceNotFoundError("Investor not found");
  }

  return investor;
}
```

The same approach applies to property listings and transactions, with appropriate resource-level permissions.

2.3 Cross-Tenant Access Error Handling

If a valid JWT authenticated under Tenant A attempts to access a resource belonging to Tenant B, the application should deny access.

Return HTTP 404 Not Found rather than 403 Forbidden.

Reason: Returning 403 can reveal that a resource exists but belongs to another tenant. Returning 404 avoids exposing information about other tenants' investors, properties, or transactions.

Error Response

```
export class ResourceNotFoundError extends Error {
  constructor(message = "Resource not found") {
    super(message);
    this.name = "ResourceNotFoundError";
  }
}

export const errorHandler = (
  error: Error,
  req: Request,
  res: Response,
  next: NextFunction
): void => {
  if (error instanceof ResourceNotFoundError) {
    res.status(404).json({
      success: false,
      message: "Resource not found",
      code: "RESOURCE_NOT_FOUND",
    });
    return;
  }
```

```
  res.status(500).json({
    success: false,
    message: "Internal server error",
    code: "INTERNAL_SERVER_ERROR",
  });
};
```

Security Logging

Cross-tenant access attempts should be logged for security monitoring and auditing.

```
logger.warn("Tenant resource access denied", {
  tenantId: req.tenantContext?.tenantId,
  investorId: req.tenantContext?.investorId,
  resourceId: requestedResourceId,
  endpoint: req.originalUrl,
  method: req.method,
  event: "CROSS_TENANT_ACCESS_ATTEMPT",
});
```

Logs must not contain JWTs, access tokens, passwords, or sensitive investor information.

Why logging matters:

- Detect potential IDOR attacks
- Identify repeated unauthorized access attempts
- Support security investigations and compliance audits
- Identify endpoints with missing authorization checks

2.4 Attack Vector — Insecure Direct Object Reference (IDOR)

Scenario

An investor authenticated under Tenant A has access to:

```
GET /api/investors/inv_101
```

The investor modifies the resource ID:

```
GET /api/investors/inv_205
```

Here, `inv_205` belongs to Tenant B. Even with a valid JWT, they must not access another tenant's resources.

How the Implementation Prevents It

1. JWT authentication establishes the trusted tenant context.
2. The service extracts `tenantId` from that context rather than accepting it from the request.
3. The repository fetches the resource using both the requested resource ID **and** authenticated tenant ID.
4. If no matching resource exists within that tenant, the application returns HTTP 404.
5. Unauthorized attempts are logged for security monitoring.

```
const investor = await investorRepository.findByIdAndTenantId(
  requestedInvestorId,
  req.tenantContext!.tenantId
);
```

```
if (!investor) {
  logger.warn("Unauthorized resource access attempt", {
    tenantId: req.tenantContext!.tenantId,
    investorId: req.tenantContext!.investorId,
    requestedInvestorId,
  });

  throw new ResourceNotFoundError();
}
```

Knowing another tenant's resource ID is insufficient to access that resource.

Security Controls Summary

Security Control	Implementation
JWT validation	Authentication middleware
Tenant identification	Verified JWT claims
Tenant context	Request-level context
Resource authorization	Service-layer validation
Cross-tenant access	HTTP 404
Security monitoring	Structured audit logging
IDOR protection	Tenant-scoped resource lookup
Financial operations	Authorization before transaction execution

Engineering decision: Use centralized authentication middleware, tenant-context propagation, and resource-level authorization inside service methods. Do not rely exclusively on middleware — a valid JWT does not automatically grant access to every resource within the platform.

Question 3 — Engineering Judgment: Fireblocks Integration

You are building the Fireblocks custody integration. On day two you discover:

- Fireblocks processes transfers asynchronously — immediate `transaction_id`, actual status (COMPLETED / FAILED / BLOCKED) arrives via webhook minutes later.
- Frontend expects a synchronous success/failure response from `POST /transactions/transfer`.
- Three days remain.

3.1 Implementation Plan for the Remaining Three Days

I would adjust the architecture rather than force asynchronous operations into a synchronous response model. Treat transfer initiation and transfer completion as two separate operations.

Transfer API — Initiate Transaction

`POST /transactions/transfer`

1. Validate the transfer request (amount, asset, destination address, investor permissions).
2. Validate the idempotency key to prevent duplicate transfer requests.
3. Create an internal transaction record with `PENDING` status.
4. Initiate the transfer through the Fireblocks SDK.
5. Store the Fireblocks `transaction_id` against the internal transaction.
6. Return **HTTP 202 Accepted** immediately.

Example Response

```
{
  "success": true,
  "message": "Transfer initiated successfully",
  "data": {
    "transaction_id": "internal_txn_123",
    "fireblocks_transaction_id": "fb_txn_456",
    "status": "PENDING",
    "message": "Your transfer is being processed"
  }
}
```

HTTP 202 indicates the request has been accepted for processing, not that the transfer has completed.

Transaction Lifecycle

```
PENDING
   |
   v
PROCESSING
   |
   +------> COMPLETED
   |
   +------> FAILED
   |
   +------> BLOCKED
```

The Fireblocks webhook endpoint receives status updates and updates the corresponding internal transaction.

Additional items to implement:

- Webhook signature verification
- Idempotent webhook event processing
- Safe transaction state transitions
- Retry handling for failed webhook processing
- Structured logging for transaction lifecycle events

Frontend Communication

A. Transaction Status API

`GET /transactions/:transactionId`

```
{
  "success": true,
  "data": {
    "transaction_id": "internal_txn_123",
    "status": "COMPLETED",
    "asset": "USDC",
    "amount": "100",
    "updated_at": "2026-11-05T12:30:00Z"
  }
}
```

The frontend can poll this endpoint until the transaction reaches a terminal state.

B. Real-time updates

If time permits, integrate WebSockets or Server-Sent Events. Given the three-day deadline, prioritize polling first (simpler to implement, test, and deliver reliably). Real-time notifications can follow later.

Three-Day Execution Plan

Day	Implementation
Day 1	Fireblocks SDK integration, transfer initiation API, idempotency, transaction persistence
Day 2	Webhook integration, signature verification, event processing, status updates
Day 3	Status API, frontend integration, reconciliation job, testing, deployment

Prioritize a reliable end-to-end transfer lifecycle over non-essential features.

3.2 Data Required for Transaction Reconciliation

Webhook delivery can fail due to network issues, application downtime, or temporary provider failures. Therefore implement a periodic reconciliation worker that checks pending transactions against the Fireblocks transaction status API.

Data to Persist

Field	Purpose
Internal transaction ID	Unique platform transaction reference
Fireblocks transaction ID	Identify the transaction with Fireblocks

Field	Purpose
Tenant ID	Identify the operator associated with the transaction
Investor ID	Identify the initiating investor
Idempotency key	Prevent duplicate transfer initiation
Asset ID	Identify the transferred token
Amount	Original transfer amount
Source vault/account	Identify the source of funds
Destination address	Identify the transfer destination
Transaction status	Track current transaction state
Fireblocks status	Store the latest provider-reported status
Transaction hash	Identify the blockchain transaction when available
Created timestamp	Track when the transfer was initiated
Last updated timestamp	Track the latest status update
Last reconciliation timestamp	Track the last provider status check
Webhook event ID	Track processed webhook events
Failure reason	Support investigation and recovery

Reconciliation Workflow

```
Scheduled Reconciliation Worker
          |
          v
Find Pending Transactions
          |
          v
Query Fireblocks API
          |
          v
Compare Transaction Status
          |
          v
Update Internal Transaction
```

Reconciliation Considerations

1. **Idempotency** — Reconciliation must not initiate another transfer; it only retrieves and synchronizes existing status.
2. **Webhook duplication** — Repeated webhook events must not cause duplicate status updates or side effects.
3. **State transitions** — A terminal state must not be overwritten by an older or stale event.
4. **Temporary failures** — If Fireblocks is unavailable, retry reconciliation with exponential backoff.
5. **Auditability** — Record status changes, provider responses, and reconciliation attempts for financial auditing.

Never automatically retry the actual transfer just because a webhook was missed. First query Fireblocks using the original transaction ID to establish the actual state.

3.3 Stakeholder Communication

Communicate the technical discovery immediately on day two. This gives the frontend team time to adapt and allows the CPO to understand product implications.

A. Message to Frontend Team

Channel: Slack
Timing: Day 2, immediately after discovery

Hi Team,

During the Fireblocks integration, I discovered an important difference in how their transfer API works.

Fireblocks processes transfers asynchronously. When we initiate a transfer, the API immediately returns a transaction ID, but the actual transaction status (COMPLETED, FAILED, or BLOCKED) is delivered later through webhooks. This can take several minutes.

Our current frontend implementation expects an immediate success/failure response, which does not align with the provider's transaction lifecycle.

Proposed API changes:

1. `POST /transactions/transfer` will return HTTP 202 Accepted with our internal transaction ID, Fireblocks transaction ID, and initial PENDING status.
2. `GET /transactions/:transactionId` will provide the latest transaction status.
3. The frontend can poll the status endpoint until the transaction reaches a terminal state.

For the current timeline, I recommend implementing polling first. We can introduce WebSockets or SSE for real-time updates in a subsequent iteration.

Please update the transfer UI to display a Processing/Pending state after successful initiation rather than immediately showing transaction success.

I'll share the finalized response contract shortly so we can align the integration.

B. Message to Sylvester (CPO)

Channel: Slack
Timing: Day 2, after the initial technical assessment

Hi Sylvester,

I identified an important integration consideration while working on the Fireblocks custody integration.

Fireblocks processes token transfers asynchronously. Our API receives an immediate transaction ID, but the actual outcome (COMPLETED, FAILED, or BLOCKED) arrives through a webhook, potentially several minutes later.

This means we cannot reliably return an immediate success/failure response from our transfer endpoint, as currently expected by the frontend.

My proposed approach:

- Return an immediate HTTP 202 response when Fireblocks accepts a transfer.
- Maintain the transaction status internally as PENDING/PROCESSING.
- Process Fireblocks webhooks to update the final transaction status.
- Provide a transaction status API so the frontend can display the actual outcome.
- Implement a reconciliation job to recover transaction statuses if a webhook is missed.

I've discussed the required API changes with the frontend team and recommended introducing a Processing state in the UI.

I can deliver the core transfer initiation, webhook processing, status tracking, and reconciliation functionality within the remaining three days.

The main product impact is a change in the user experience: investors will see that their transfer is processing rather than receiving an immediate success confirmation. This is necessary to ensure we never report a transfer as successful before Fireblocks confirms its completion.

I'll prioritize transaction reliability, idempotency, and accurate status reporting over attempting to preserve an incorrect synchronous success/failure contract.

Final Engineering Decisions

Area	Decision
Transfer initiation	HTTP 202 Accepted
Initial transaction status	PENDING
Transaction processing	Asynchronous
Status updates	Fireblocks webhooks
Frontend communication	Polling initially; WebSockets/SSE later
Duplicate transfers	Idempotency key
Duplicate webhooks	Event-level idempotency
Missed webhooks	Scheduled reconciliation
Failed transactions	Persist failure reason
Financial safety	Never assume success before provider confirmation

Overall approach: Do not compromise the correctness of a financial transaction to accommodate a synchronous frontend contract. Adapt the API contract to Fireblocks' actual behavior, communicate the change immediately, and deliver a reliable asynchronous transaction lifecycle within the remaining three days.

AI Usage Review

1. AI Tools Used

I used ChatGPT as an AI assistant throughout this assessment. Approximately 60–65% of the initial documentation and written responses were developed with AI assistance. My primary use was to structure technical explanations, organize paragraphs, and document implementation approaches based on discussions with the AI agent.

2. Key Prompts and Usage

Key prompts involved designing OAuth token management, webhook processing with idempotency, multi-tenant authorization, retry mechanisms, and asynchronous Fireblocks transaction handling.

I also discussed advanced engineering concepts such as concurrency management, atomic database operations, caching strategies, and transaction reconciliation.

3. What AI Got Right

AI was particularly helpful in establishing the initial application flow, identifying major components, defining service responsibilities, and structuring technical explanations. It provided a good starting point for standard implementation patterns such as token caching, webhook handling, exponential backoff, and asynchronous processing.

4. What AI Got Wrong

The main limitation was its lack of complete application-specific context. While AI suggested technically valid approaches, it did not always identify the most suitable, mature, well-documented, and thoroughly tested libraries for production use.

It also required additional guidance when dealing with advanced concepts such as consistent hashing, concurrency, database transaction boundaries, atomic operations, and determining where prefetching and caching would be beneficial.

Some suggestions were generic and did not fully account for distributed execution, race conditions, or production failure scenarios.

5. What I Changed

I critically reviewed the generated responses and refined them using my engineering judgment. I challenged assumptions, evaluated implementation approaches, considered concurrency and transaction boundaries, and emphasized idempotency and failure recovery.

I also ensured that database operations were considered in terms of business consistency rather than treating every operation as an isolated query.

Overall, I used AI as a technical discussion and documentation assistant rather than an authoritative source of engineering decisions. I retained responsibility for evaluating suggestions, correcting shortcomings, and ensuring the final solutions were technically appropriate.